PCC104 - Projeto e Análise de Algoritmos

Prof. Rodrigo Silva

20 de março de 2026

UNIVERSIDADE FEDERAL DE OURO PRETO

Outline

Algoritmo

Introdução à Análise de Algoritmos

Algoritmo

- Procedimento computacional
- Bem definido
- Toma algum valor ou conjunto de valores de entrada
- Produz algum valor ou conjunto de valores de saída

Procedimento Computacional

- Sequência de passos bem definidos (não ambíguos)
- Cada passo descreve uma operação elementar
- Cada passo é simples e executável em tempo finito

Algoritmo

- Procedimento para resolver um problema computacional

Problema Computacional

Definição

Um problema computacional é a especificação de uma tarefa que define, para cada valor de entrada, quais valores de saída são considerados corretos.

Exemplo: Ordenação

Entrada: uma sequência de números

$$(a_1, a_2, \ldots, a_n)$$

Saída: uma permutação da sequência tal que

$$a_1 \leq a_2 \leq \cdots \leq a_n$$

Instância de um Problema

Definição

Uma instância de um problema é um conjunto específico de valores de entrada para um problema computacional.

Exemplo: Problema de Ordenação

Problema: ordenar uma sequência de números.
Instância:

$$(8, 3, 5, 2, 9)$$

Problema, Instância e Algoritmo

Conceitos

- **Problema** $\rightarrow$ classe de tarefas
- **Instância** $\rightarrow$ caso específico
- **Algoritmo** $\rightarrow$ método para resolver

Exemplo

Problema: ordenação

$$\Downarrow$$

Instância: $(8, 3, 5, 2, 9)$

$$\Downarrow$$

Algoritmo

$$\Downarrow$$

Saída: $(2, 3, 5, 8, 9)$

Algoritmo Correto

Definição

Um algoritmo é dito correto se, para toda instância válida do problema, ele produz uma saída que satisfaz a especificação do problema em tempo finito.

Ordenação por Inserção (Insertion Sort)

Problema de Ordenação

Entrada: Uma sequência de n números

$$\langle a_1, a_2, \ldots, a_n \rangle$$

Saída: Uma permutação (reordenação) da sequência de entrada

$$\langle a'_1, a'_2, \ldots, a'_n \rangle$$

tal que

$$a'_1 \leq a'_2 \leq \cdots \leq a'_n$$

Objetivo

Reordenar os elementos da entrada em ordem não decrescente.

Ordenação por Inserção

Insertion-Sort(A)

```
for j ← 2 to A.length do
    key ← A[j]
                ▷ Insere A[j] na sequência ordenada A[1 ... j − 1]
    i ← j − 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]
        i ← i − 1
    end while
    A[i + 1] ← key
end for
```

Exemplo : $[5, 2, 4, 6, 1, 3]$

Corretude da ordenação por inserção

Invariante de Laço

Uma **invariante de laço** é uma propriedade que deve ser verdadeira:

- **Inicialização:** antes da primeira iteração do laço;
- **Manutenção:** antes de cada iteração, permanece verdadeira após a execução do corpo do laço;
- **Término:** ao final do laço, fornece uma propriedade útil para demonstrar a correção do algoritmo.

Uso

Invariantes de laço são utilizadas para demonstrar a **correção** de algoritmos iterativos.

Correção do Insertion Sort

Invariante de Laço

No início de cada iteração do laço **for** (linha 1), o subarray

$$A[1 \ldots j-1]$$

consiste nos elementos originalmente em $A[1 \ldots j-1]$, porém em ordem não decrescente.

Prova

Inicialização: Para $j = 2$, o subarray $A[1]$ contém apenas um elemento, logo está ordenado.

Manutenção: A cada iteração, o elemento $A[j]$ é inserido na posição correta em $A[1 \ldots j-1]$, produzindo $A[1 \ldots j]$ ordenado.

Término: Quando o laço termina ($j = A.\text{length} + 1$), temos

$$A[1 \ldots A.\text{length}]$$

ordenado.

O que é Análise de Algoritmos?

Definição

A **análise de algoritmos** consiste em prever os recursos computacionais necessários para executar um algoritmo, tipicamente em função do tamanho da entrada n.

Por que analisar algoritmos?

- Comparar diferentes algoritmos para o mesmo problema;
- Prever desempenho para entradas grandes;
- Compreender como o tempo de execução cresce com n;
- Guiar o projeto de algoritmos mais eficientes.

Desafio

Para analisar um algoritmo, precisamos de um modelo que defina:

- O que é uma operação;
- Quanto custa cada operação.

Modelo de Computação (RAM -*Random-access Machine*)

Na análise de algoritmos, assumimos o modelo RAM, no qual:

- As instruções são executadas sequencialmente;
- Cada operação básica (atribuição, soma, comparação, acesso a memória) custa tempo constante;
- O acesso à memória é uniforme (tempo constante para qualquer posição).

Cuidados

- O custo constante pode variar entre operações (ex.: divisão vs. soma);
- Ignoramos efeitos de hardware (cache, paralelismo, I/O);
- Assumimos que os dados cabem na memória principal.

Limitações

- Não modela hierarquia de memória;
- Não captura custo de operações com números muito grandes;
- Pode não refletir desempenho real em sistemas modernos.

Análise do Insertion Sort

Por que analisar?

- Determinar o custo computacional do algoritmo
- Comparar diferentes algoritmos para o mesmo problema
- Prever comportamento em diferentes tamanhos de entrada

Modelo de Análise (CLRS)

- Modelo **RAM** (Random Access Machine)
- Cada operação elementar tem custo constante
- Tempo total = número de execuções de cada linha

Ideia central

- O tempo depende do número de deslocamentos realizados
- Esse número varia conforme a **ordem da entrada**

Especificando a Análise

- Modelo RAM
- Tamanho da entrada?

Insertion Sort: Custo e Frequência

Ordena-Por-Inserção(A, n)

1. **for** $i = 2$ **to** n
2. chave $= A[i]$
3. $j = i - 1$
4. **while** $j > 0$ **and** $A[j] >$ chave
5. $A[j+1] = A[j]$
6. $j = j - 1$
7. $A[j+1] =$ chave

Custo: n^o Vezes

- $c_1 : n$
- $c_2 : n - 1$
- $c_3 : n - 1$
- $c_4 : \sum_{i=2}^{n} t_i$
- $c_5 : \sum_{i=2}^{n} (t_i - 1)$
- $c_6 : \sum_{i=2}^{n} (t_i - 1)$
- $c_7 : n - 1$

Observação

t_i é o número de vezes que o teste do `while` é executado para cada i.

Análise Insertion Sort

Tempo Total

$$T(n) = c_1 n - + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^{n}(t_i)$$
$$+ c_5 \sum_{i=2}^{n}(t_i - 1) + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

Observação

Mesmo para entradas de mesmo tamanho o tempo de execução pode sdepender de qual entrada desse tamanho é dada.

Análise Insertion Sort - Entrada ordenada

Tempo Total

$$T(n) = c_1 n - + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^{n}(t_i)$$
$$+ c_5 \sum_{i=2}^{n}(t_i - 1) + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

$T(n) = an + b$ (Tempo linear)

Análise Insertion Sort - Ordem Decrescente

Tempo Total

$$T(n) = c_1 n - +c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^{n}(t_i)$$
$$+ c_5 \sum_{i=2}^{n}(t_i - 1) + c_6 \sum_{i=2}^{n}(t_i - 1) + c_7(n-1)$$

$T(n) = an^2 + bn + c$ (Tempo quadrático)

Casos de Análise de Algoritmos

Melhor Caso

- Menor tempo de execução para entradas de tamanho n
- Ocorre quando a entrada favorece o algoritmo
- Formalmente:

$$T_{\text{best}}(n) = \min_{\text{entradas de tamanho } n} T(n)$$

Casos de Análise de Algoritmos

Pior Caso

- Maior tempo de execução para entradas de tamanho n
- Garante limite superior de desempenho
- Formalmente:

$$T_{\text{worst}}(n) = \max_{\text{entradas de tamanho } n} T(n)$$

Casos de Análise de Algoritmos

Caso Médio

- Tempo esperado sobre todas as entradas de tamanho n
- Depende de uma distribuição de probabilidade
- Formalmente:

$$T_{\text{avg}}(n) = \mathbb{E}[T(n)]$$

Por que analisar o pior caso?

Garantia de Desempenho

- Fornece um limite superior para o tempo de execução
- Garante que o algoritmo nunca será pior do que isso
- O caso médio é quase tão ruim quanto o pior caso

Independência da Entrada

- Não depende de hipóteses sobre a distribuição dos dados
- Evita suposições difíceis de justificar na prática

Robustez

- Essencial em sistemas críticos (tempo real, segurança, etc.)
- Protege contra entradas adversariais

Caracterização dos Tempos de Execução

Caracterização dos Tempos de Execução

$$
\begin{aligned}
T(n) = & \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right) n^2 \\
& + \left(c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7\right) n \\
& - (c_2 + c_3 + c_4 + c_7)
\end{aligned}
$$

Por que não contar exatamente o número de operações?

Constantes e Detalhes Irrelevantes

- Diferenças como:

$$T(n) = 3n^2 + 10n + 5 \quad \text{vs} \quad T(n) = n^2$$

- Para n grande, o termo dominante é o mesmo

Baixa Escalabilidade da Análise Exata

- Expressões exatas são complexas e pouco informativas
- Difíceis de comparar entre algoritmos

Ideia Central

- Queremos capturar o comportamento para n grande
- Por isso usamos **taxa de crescimento** e **notação assintótica**

Notação O (Big-O)

Definição Intuitiva

- A notação O caracteriza um **limite superior** assintótico
- Indica que uma função não cresce mais rápido que uma certa taxa
- Baseia-se no **termo dominante** (maior ordem)

Exemplo

$$T(n) = 7n^3 + 100n^2 - 20n + 6$$

- Termo dominante: n^3
- Logo: $T(n) = O(n^3)$

Notação O (Big-O)

Observação Importante

- Também podemos escrever:

$$T(n) = O(n^4),\ O(n^5), \ldots$$

- Pois a função cresce mais lentamente que essas taxas
- Em geral: $T(n) = O(n^c)$ para qualquer $c \geq 3$

Mensagem-chave

A notação O fornece um **limite superior**, não necessariamente o mais justo.

Notação Ω (Big-Omega)

Definição Intuitiva

- A notação Ω caracteriza um **limite inferior** assintótico
- Indica que uma função cresce **pelo menos tão rápido** quanto uma certa taxa
- Foca no comportamento para n **grande**

Exemplo

$$T(n) = 7n^3 + 100n^2 - 20n + 6$$

- Termo dominante: n^3
- Logo: $T(n) = \Omega(n^3)$

Notação Ω (Big-Omega)

Observação Importante

- Também podemos escrever:

$$T(n) = \Omega(n^2),\ \Omega(n), \ldots$$

- Pois a função cresce pelo menos tão rápido quanto essas taxas
- Em geral: $T(n) = \Omega(n^c)$ para qualquer $c \leq 3$

Interpretação

- $\Omega(\cdot)$ fornece um **limite inferior**
- Descreve o **melhor crescimento possível** do algoritmo

Notação Θ (Big-Theta)

Definição Intuitiva

- A notação Θ caracteriza um **limite assintótico justo**
- Indica que a função cresce **exatamente** a uma certa taxa
- Considera limites superior e inferior simultaneamente

Interpretação

- A função está limitada por constantes:

$$c_1 f(n) \leq T(n) \leq c_2 f(n)$$

- Para n suficientemente grande
- Os fatores c_1 e c_2 não precisam ser iguais

Notação Θ (Big-Theta)

Caracterização

- Se:

$$T(n) = O(f(n)) \quad \text{e} \quad T(n) = \Omega(f(n))$$

- Então:

$$T(n) = \Theta(f(n))$$

Exemplo

$$T(n) = 7n^3 + 100n^2 - 20n + 6$$

- $T(n) = O(n^3)$ e $T(n) = \Omega(n^3)$
- Logo: $T(n) = \Theta(n^3)$

Mensagem-chave

$\Theta(\cdot)$ descreve a **taxa de crescimento exata** do algoritmo.

Notação O — Definição Formal

Ideia

- A notação O descreve um **limite assintótico superior**
- Fornece um limite dentro de um **fator constante**

Definição Formal

Para uma função $g(n)$, definimos:

$$O(g(n)) = \{f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tais que } 0 \leq f(n) \leq c\, g(n),\ \forall n \geq n_0\}$$

Interpretação

- A função $f(n)$ não cresce mais rápido que $g(n)$
- A desigualdade vale a partir de algum ponto n_0

Exemplo: Prova usando a definição formal de O

Objetivo

Mostrar que:

$$f(n) = 3n^2 + 5n + 2 \text{ é } O(n^2)$$

Passo 1: Majorar a função

Para $n \geq 1$:

$$3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$$

Passo 2: Escolha das constantes

- $c = 10$
- $n_0 = 1$

Logo:

$$0 \leq f(n) \leq 10n^2 \quad \forall n \geq 1$$

Conclusão

Pela definição formal, $f(n) = O(n^2)$.

Notação Ω — Definição Formal

Ideia

- A notação Ω descreve um **limite assintótico inferior**
- Garante crescimento mínimo dentro de um **fator constante**

Definição Formal

Para uma função $g(n)$, definimos:

$$\Omega(g(n)) = \{f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tais que } 0 \leq c\, g(n) \leq f(n),\ \forall n \geq n_0\}$$

Interpretação

- A função $f(n)$ cresce pelo menos tão rápido quanto $g(n)$
- A desigualdade vale a partir de algum ponto n_0

Notação Θ — Definição Formal

Ideia

- A notação Θ descreve um **limite assintótico justo**
- Combina limite superior e inferior

Definição Formal

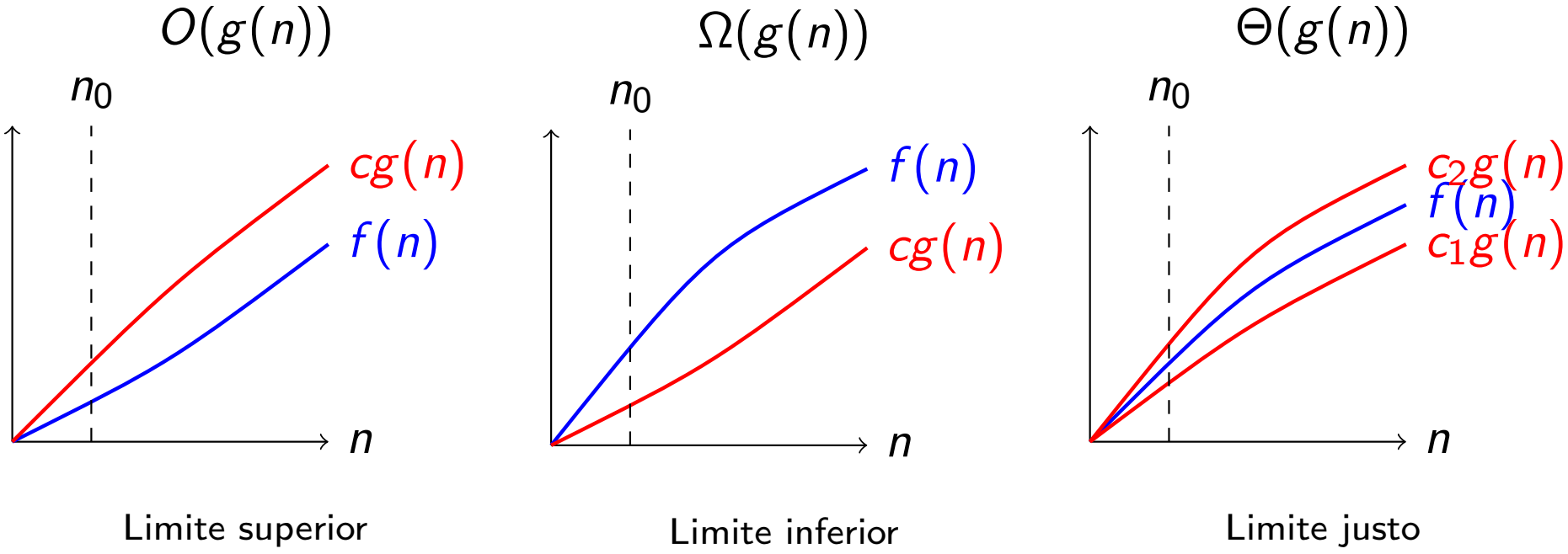
Para uma função $g(n)$, definimos:

$$\Theta(g(n)) = \{f(n) : \exists\, c_1, c_2 > 0,\ \exists\, n_0 > 0 \text{ tais que}$$
$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n),\ \forall n \geq n_0\}$$

Interpretação

- $f(n)$ está limitada superior e inferiormente por $g(n)$
- Descreve a **taxa de crescimento exata**

Notações Assintóticas: Interpretação Gráfica



Mensagem-chave

Para $n \geq n_0$: O limita acima, Ω abaixo e Θ entre dois limites.

Comparação de Funções Assintóticas

Hipótese

Considere $f(n)$, $g(n)$ e $h(n)$ funções assintoticamente positivas.

Transitividade

- Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$
- Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$
- Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$
- Se $f(n) = o(g(n))$ e $g(n) = o(h(n))$
- Se $f(n) = \omega(g(n))$ e $g(n) = \omega(h(n))$

Ideia

As relações assintóticas preservam a ordem de comparação entre funções.

Comparação de Funções Assintóticas

Hipótese

Considere $f(n)$, $g(n)$ e $h(n)$ funções assintoticamente positivas.

Transitividade

- Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$, então $f(n) = \Theta(h(n))$
- Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$, então $f(n) = O(h(n))$
- Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$, então $f(n) = \Omega(h(n))$
- Se $f(n) = o(g(n))$ e $g(n) = o(h(n))$, então $f(n) = o(h(n))$
- Se $f(n) = \omega(g(n))$ e $g(n) = \omega(h(n))$, então $f(n) = \omega(h(n))$

Ideia

As relações assintóticas preservam a ordem de comparação entre funções.

Comparação de Funções Assintóticas

Reflexividade

$$f(n) = \Theta(f(n)), \qquad f(n) = O(f(n)), \qquad f(n) = \Omega(f(n))$$

Simetria

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Simetria de Transposição

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \iff g(n) = \omega(f(n))$$

Mensagem-chave

Θ é simétrica; O e Ω são relações transpostas uma da outra.